# Laboratory Work 2: A Coffee Shop Server

**Goal.** Build the server of a coffee shop, using TCP sockets directly. Clients order products; the server takes them from its stock, prepares the orders with a fixed number of workers, and tells each client when its order is ready. A provider brings new products. Serve many clients at the same time, protect the stock they share, and keep the server running when clients fail.

**Keywords:** TCP sockets · client–server · producer–consumer · worker threads · shared state · race condition

## 1 Requirements

- Any programming language with a socket API.
- Use the socket API directly: the equivalents of socket, bind, listen, accept, connect, send, receive and close (in C#, the `Socket` class; in Java, `Socket` and `ServerSocket`; in Python, the `socket` module). Wrappers above it are not allowed: `TcpListener`, `TcpClient` and `NetworkStream` in C#, `socketserver` in Python, and any library or framework for HTTP, WebSocket, RPC or messaging.
- Three roles: the server, the client and the provider. The client and the provider can be one program that starts in either role. Both are interactive console programs. The port of the server, and the address and port they connect to, are command-line parameters.
- The server writes a log line for every connection, every order (accepted, refused, ready), every delivery and every error, with the endpoint of the client.
- A Git repository with the source code (for example, on GitHub or GitLab) and instructions to build and run the programs; its link is sent to the teacher before the defense.

## 2 The coffee shop

The shop sells three products. Each one takes a fixed time to prepare:

| Product | Preparation time |
|---|---|
| coffee | 4 s |
| tea | 3 s |
| croissant | 2 s |

When the server starts, it puts a random amount from 10 to 50 of each product into its stock.

An order is a list of products with their quantities. Its preparation time is the sum of the times of all its items: 2 croissants, 1 tea and 1 coffee take $2 \cdot 2 + 3 + 4 = 11$ seconds. The preparation is simulated by waiting.

The server prepares the orders with *workers*: it creates one worker for each logical processor (CPU core) of its machine. A worker prepares one whole order at a time. When all the workers are busy, new orders wait in a queue, in the order in which they arrived.

## 3 Task 1: One client

*Goal.* The menu, the stock and one order at a time.

A client connects and can ask for the menu: the products and how many of each are in stock. It can then order. If the order asks for more of a product than the stock has, it is refused, and the client is told how many are left. Otherwise the server takes the products from the stock at once, answers the client right away with the number of the order and its preparation time, and, when the order has been prepared, tells the client that it is ready. An unknown product or a quantity that is not a positive number gets an error message.

*Questions for the defense.*

1. How does your code cut the received bytes into messages? What does it do when a message arrives in two pieces, or two messages arrive in one receive?
2. Why does the server answer at once, and tell the client later that the order is ready? What would the client see otherwise?

## 4 Task 2: Many clients

*Goal.* Serve many clients at the same time and protect the stock they share.

Clients connect at any time, and each of them can order while the orders of others are being prepared. The server creates its workers as described above; a free worker takes the oldest waiting order. A client that has ordered can go on asking for the menu or ordering again while it waits. Two clients that order the last croissant at the same moment must not both get it.

Show it at the defense with more clients than workers.

*Questions for the defense.*

1. Which data is shared between threads in your server, and how is each of them protected?
2. Two clients order the last croissant at the same moment. Show, step by step, what could go wrong without your protection.
3. A worker tells a client that its order is ready while the thread of that client is reading from the same connection. What must you take care of?
4. Why one worker per CPU core? The workers spend their time waiting: what would happen with twice as many?

## 5 Task 3: The provider

*Goal.* A second kind of connection, and messages that the server sends on its own.

The provider connects to the server like a client, but tells the server that it is the provider. Only the provider can bring products: it sends a delivery, for example 20 coffees and 10 croissants, and the products are added to the stock at once. A client that tries to deliver gets an error message.

When a product falls below 5 in the stock, the server alerts the provider at once, without waiting for a request. If the provider is not connected, it gets the alert when it connects.

*Questions for the defense.*

1. How does your server know which connection is the provider?
2. A delivery and an order change the stock at the same moment. Show that no change is lost.
3. Which thread sends the alert to the provider, and when?

## 6 Task 4: When things go wrong

*Goal.* Keep the shop open for everybody else when one connection fails.

A client can disappear at any moment: it closes the connection, its process is killed, or its network is gone. The server notices it and cancels all its orders: those that still wait in the queue, and the one that a worker may already be preparing, in which case the worker stops. In every case the products go back to the stock, but whatever was prepared is lost: the next order that takes them waits for their whole preparation again. When the provider disappears, the shop goes on with the stock it has.

Every invalid input gets an error message: an unknown command, a missing or wrong parameter, a message that is too long, or random bytes. The server never crashes and never blocks other clients because of it. The client and the provider show every message from the server as soon as it arrives, even while their user is typing, and end cleanly when the server disappears.

*Questions for the defense.*

1. How does your server notice that a client has disappeared? Which kind of disappearance does it notice at once, and which one not?
2. A client disappears while a worker is preparing its order. How does the worker find out and stop, and how do the products get back to the stock?
3. Why must the client receive and read the keyboard at the same time? How does yours do it?

## 7 Control questions

1. What is the difference between the listening socket and the sockets returned by accept? How many of each does your server have with five clients and the provider?
2. Why is one send not one receive? How does your protocol deal with it?
3. What does it mean when receive returns 0? And when it fails because the connection was reset?
4. Where in your server is a producer–consumer pattern? Name the producers, the consumers and the queue between them.
5. What does it cost when a thread waits for a client? What would change with 1000 clients?
6. Explain step by step a race condition that could happen on your server without its locks.
7. What identifies one TCP connection? How can five clients on the same machine all be connected to the same port of the server?

## 8 Bonus

Any of the following earns bonus points at the defense:

- the shop working between two computers in the same network;
- the stock saved by the server, so that it is kept when the server restarts;
- the server and the clients written in two different programming languages, working together.

## 9 Grading

Each grade requires everything above it in Table 1. A task counts only if it works at the defense, and you can explain its code and answer its questions (code understanding).

**Table 1.** Requirements for each grade.

| Grade | Completed |
|---|---|
| 5 | Task 1 |
| 6 | Task 2 |
| 7 | Task 3 |
| 8 | Task 4 |
| 9 | At least half of the control questions, answered at the defense |
| 10 | All control questions, answered at the defense |

*Would you like a cup of tea?*
`https://www.youtube.com/watch?v=r799U_-jAnk`